Minecraft Server Distributions and Architecture Guide

January 22, 2026

Contents

1 Introduction

This document provides a comprehensive overview of **Minecraft server distributions**, their relationships, and the fundamental differences between plugin-based and mod-based architectures. The content is organized by abstraction level, starting with basic server concepts and progressing to advanced networking configurations.

The document is structured to guide readers from fundamental concepts to advanced configurations, ensuring a clear understanding of the technical landscape before making **architectural decisions**.

2 What is a Minecraft Server?

A **Minecraft server** is a Java process that runs the game logic that players connect to. It hosts worlds, enforces rules, executes plugins or mods, and handles all player network traffic.

2.1 Main Components

A typical Minecraft server consists of:

- **Java Process**: Executes the server JAR file (vanilla, Paper, Fabric, Forge, etc.)
- **World Data**: Region files and player data stored on disk
- **Configuration Files**: `server.properties` and plugin/mod configuration files
- **Network Listener**: TCP port 25565 for Java Edition (Bedrock Edition uses different ports)
- **Add-ons**: Plugins (for Paper/Spigot) or mods (for Fabric/Forge) installed into the server process

2.2 Key Characteristics

- **Single Process Model**: Changes affect all connected players, providing centralized control
- **Server Type Determines Add-ons**: The server type (**Paper** vs **Fabric** vs **Forge**) determines what add-ons you can run
- **Performance Varies**: Performance and memory usage depend on the implementation and installed add-ons

3 Family Tree of Server Variants

Understanding the evolution and relationships between different server implementations is crucial for making informed choices about **compatibility** and **features**.

3.1 Plugin Lineage (API + Server Implementations)

The **plugin-based server lineage** represents a progressive evolution where each step adds more performance and features while maintaining **compatibility** with previous versions.

3.1.1 Bukkit (The API)

Bukkit is the foundational API—the "language" that allows third-party tools to communicate with the Minecraft server. It defines the interface that plugins use, but it does not run by itself; it requires an **implementation**.

3.1.2 Spigot (The Performance Fork)

Spigot is a modified version of the original server code that implements the **Bukkit API**. It adds performance settings and efficiency improvements over the vanilla server implementation.

3.1.3 Paper (The Standard)

Paper is a fork of Spigot. It is even faster than Spigot and fixes "vanilla" bugs that Spigot misses. From a software architecture perspective, Paper is built on top of Spigot's codebase as a fork, meaning it replaces Spigot entirely rather than depending on it as an external library or runtime requirement. Paper has become the de facto standard for plugin-based servers.

Compatibility: Plugins written to the **Bukkit/Spigot API** generally run on both Spigot and Paper (**Paper is a superset**).

```
Bukkit (API)
  \- Spigot (implements Bukkit)
      \- Paper (fork of Spigot; extra performance/fixes)
```

3.2 Mod Loaders (Modify Game Code Directly)

Mod loaders operate differently from the plugin lineage. They modify or extend the game code itself rather than using a **layered API approach**.

3.2.1 Forge: The Classic

Forge is the classic mod loader. It is heavy, complex, and supports massive, old-school modpacks (like RLCraft). It is slow to load and heavy on resources, but it has the broadest mod ecosystem.

3.2.2 Fabric: The Modern

Fabric is the modern mod loader. It is extremely lightweight and fast. It is the best choice for OCI VMs because it uses very little RAM. It is famous for performance mods like Lithium.

3.2.3 Quilt: The Community Fork

Quilt is a newer fork of Fabric. It aims to be more community-driven and is compatible with most Fabric mods.

Compatibility: Mods are written for a specific loader; **Fabric mods** do not run on **Forge** and vice versa (unless a wrapper exists).

```
Vanilla client/server
  +- Forge (mod loader)
  +- Fabric (lightweight mod loader)
  \- Quilt (Fabric fork)
```

3.3 Key Differences and Trade-offs

- **Plugin Servers** (**Paper**/**Spigot**) let any vanilla Java client join **without client mods**
- **Mod Servers** change or extend game code and typically require clients to install the same mods (unless the mods are server-side-only)
- **Incompatibility**: You cannot drop Fabric mods into a Paper server directly because they target a different **runtime/instrumentation model**

4 Plugins vs. Mods: A Detailed Comparison

The reason you cannot simply "add" Fabric to Paper is that they use completely different "engines" to modify the game. Understanding these differences is essential for making **informed architectural decisions**.

4.1 Functional Comparison

Feature	Plugins (Paper/Spigot)	Mods (Fabric/Forge)
Location	Server-side ONLY	Server-side AND Client-side
Capabilities	Hook into existing server APIs, change gameplay logic, add commands, manage players, economy, GUIs on server-side	Can add new blocks/items/entities, change rendering, add client-side systems, or alter base game mechanics more deeply
New Content	Cannot add native engine-level blocks/items/mobs[1]	Can add anything (new planets, machines, magic)
Client Requirements	Any "vanilla" client can join	Clients usually must install matching mods (unless the mod is server-side-only or a protocol bridge exists)
Complexity/Performance	Generally simpler to manage and more compatible with existing server tooling	More powerful but require more careful dependency/version management; some mod loaders are heavier (Forge) while others are lightweight (Fabric)
Architecture	Use a stable API (**Bukkit**) layered on a server implementation	Alter or instrument the game code itself (bytecode/patching systems)

4.2 When to Use Which

4.2.1 Use Plugins (Paper) if:

- You want maximum compatibility with **vanilla clients**
- You need quick setup and many **server-side features** (economies, minigames)
- You expect **vanilla Java clients** to join without modifications
- You prioritize simplicity and ease of management

[1] Plugins can simulate custom content via the "content illusion system" methodology using resource packs and entity/item manipulation. See Section 8 for implementation details.

4.2.2 Use Mods (Fabric/Forge) if:

- You need new content (blocks, machines, entities) that plugins cannot provide
- You want low-level performance mods that change engine behavior
- You control the client environment (or the mod is server-side-only)
- You need capabilities beyond what the **plugin API** can offer

5 Proxy Architecture and Bedrock Compatibility

Proxy servers enable advanced networking configurations, including allowing Bedrock Edition players to join **Java servers** and mixing different server types behind a single front-end.

5.1 Purpose

Proxy architecture allows:

- Bedrock Edition players (phones, consoles, Windows 10/11 Bedrock) to join a **Java server network**
- Mixing different server types (Paper, Fabric, etc.) behind a single proxy
- Centralized routing and protocol translation

5.2 Key Components

5.2.1 Proxy (Velocity, BungeeCord)

A network front-end that routes players to backend servers. It presents a single public endpoint and can connect multiple backend servers (Paper, Fabric, etc.). The proxy handles initial connections and forwards players to appropriate backend servers.

Forwarding Protocol Differences: **Velocity** uses the Modern forwarding protocol, while **BungeeCord** uses the Legacy forwarding protocol. This distinction is important when configuring Fabric backends, as different forwarding protocols have different compatibility requirements (see Section 5.2.4).

5.2.2 Geyser

A protocol translator that allows **Bedrock clients** to connect to **Java servers**. **Geyser** can run on the proxy or on individual backend servers. When on the proxy, it centralizes protocol translation before forwarding to backends.

Note for Custom Content Systems: For high-interactivity custom content (such as the input engine described in Section 8, Phase 7), having **Geyser on the backend** or using the **Geyser API** is often necessary to handle Bedrock-specific player metadata (device OS, input type) and Bedrock-specific UI forms effectively. Proxy-based Geyser may limit backend plugins' access to this metadata.

5.2.3 Floodgate

An authentication layer that lets Bedrock players join without Mojang accounts. It pairs with Geyser and must be configured to ensure Bedrock players get stable UUIDs and proper permissions on backend servers.

5.2.4 FabricProxy-Lite

A Fabric-side mod that enables a **Fabric backend** to maintain UUIDs/skins and communicate correctly with **Velocity** when not using Paper on the backend. This is necessary because **Velocity** uses the Modern forwarding protocol, which **Fabric** does not support natively.

Note: If using **BungeeCord** (which uses the Legacy forwarding protocol), the requirements differ and FabricProxy-Lite may not be needed, depending on your specific setup.

5.3 Practical Configuration Options

5.3.1 Simple: Paper Backend + Velocity Proxy

- Run Velocity as the public endpoint
- Backend servers: Paper instances
- Install **Geyser and Floodgate** on the Velocity proxy (works well; proxy handles **Bedrock translation**)
- **Benefit**: Easiest setup, broad compatibility, **vanilla Java clients** unaffected
- **Note**: For custom content systems requiring Bedrock-specific features (see Section 8), consider placing **Geyser on the backend** or using the **Geyser API** for full metadata access

5.3.2 Mixed: Velocity Proxy + Fabric Backend

- Use Velocity as the proxy
- Backend: Fabric server for server-side Fabric mods
- Install FabricProxy-Lite on the Fabric backend to ensure Velocity and backend share UUID/skin mapping and correct player handling
- Geyser and Floodgate can be on the Velocity proxy OR moved to the Fabric backend (both work; proxy centralizes translation)
- **Benefit**: Lets you run Fabric server-side mods while still using **Velocity** as central router and serving **Bedrock clients**
- **Recommendation**: For custom content systems (Section 8), prefer **Geyser on the backend** to enable full access to Bedrock-specific player metadata and UI features

5.4 Placement and Behavior Notes

- **Geyser on Proxy**: Proxy does protocol translation for **Bedrock** before forwarding to backend; backend sees connections like **Java clients**. This centralizes translation but may limit backend access to Bedrock-specific metadata (device type, input method).
- **Geyser on Backend**: Translation happens on the server that hosts the world; proxy acts only as a router. This enables full access to **Geyser API** features for custom content systems requiring Bedrock-specific UI forms or player metadata.
- **Floodgate Configuration**: Must be configured to ensure Bedrock players get stable UUIDs and proper permissions on backend

5.5 Trade-offs and Considerations

- Running translation on the proxy centralizes logic (simpler if you have many backends)
- Some Fabric mods may expect certain networking behaviors; FabricProxy-Lite (or similar) smooths differences between proxy and Fabric backend
- Cross-server features (like Bungee/Velocity plugin expectations) are easiest when backends are Paper/Spigot; mixed networks require extra bridging/config

6 Summary and Recommendations

Since you are using **Geyser** to let Bedrock players join, you are effectively "locked" to a specific type of modding:

- **Stick with Paper** if you only want commands, economy, and performance improvements. This is the simplest and most compatible option, especially when using a **proxy architecture**.
- **Switch to Fabric** only if you want server-side mods (like Lithium or Ledger) that offer even better performance than Paper but do not require clients to install anything. Remember to install **FabricProxy-Lite** when using Velocity as your proxy.

7 Conclusion

The choice between plugin-based and mod-based architectures depends on your specific requirements:

- **Plugin-based servers** (**Paper**/**Spigot**) offer simplicity, broad compatibility, and ease of use. They are ideal for most use cases, especially when serving **vanilla clients**.
- **Mod-based servers** (**Fabric**/**Forge**) offer greater flexibility and performance optimizations but require more technical knowledge and may have **compatibility constraints**.
- **Proxy architecture** enables advanced configurations, including **Bedrock compatibility** and mixing server types, but adds complexity to your setup.

Consider your use case, technical expertise, performance requirements, and **client compatibility needs** when making your decision. Start simple with **Paper** if you are unsure, and migrate to **Fabric** only if you have specific requirements that plugins cannot meet.

8 Implementation Plan: Paper-Based Custom Content System

This section presents a comprehensive implementation plan for achieving mod-like experiences using **Paper and plugins** without requiring client-side modifications. This approach is precisely how large **cross-play servers** (**Java** + **Bedrock**) deliver custom content while maintaining zero client setup requirements.

8.1 High-Level Architecture

The fundamental principle is building a *content illusion system* rather than real blocks. The architecture consists of:

```
Paper Server
  \- Core Gameplay Plugin (your code)
      \- Custom item system
      \- Fake block / furniture system
      \- Entity-based animals & fish
      \- Interaction engine (click/sneak/etc.)
      \- GUI framework

  \- Resource Pack (mandatory)
      \- Textures
      \- Models
      \- Sounds
      \- Fonts

  \- Geyser + Floodgate
      \- Bedrock forms / UI mapping

  \- Optional helper plugins
      \- ProtocolLib
      \- Citizens
      \- ItemsAdder-like concepts (you reimplement)
      \- ModelEngine-like concepts
```

8.2 Core Methodology

Rule #1: Server owns logic, client owns visuals.

- **Server decides what happens**: All game logic and state management occurs **server-side**
- **Resource pack decides how it looks**: Visual representation is handled entirely through **resource packs**
- **Client stays vanilla**: No client-side modifications are required

Everything built follows this fundamental separation of concerns.

8.3 Step-by-Step Implementation Plan

8.3.1 Phase 1: Resource Pack (Foundation)

Priority: Execute this phase first. Everything else depends on the resource pack.
Content Requirements:

- Custom fish models
- Animal variants (reskins)

- Furniture models
- Machine models
- GUI icons
- Fonts (Unicode font mapping)
- Sounds (ambient, machine hum, UI click)

Key Techniques:

- `CustomModelData` for item differentiation
- Item-based models (stick, carrot_on_a_stick, paper)
- Block-model-with-entity illusion (via item frames or entities)

Folder Structure:

```
assets/minecraft/models/item/
assets/minecraft/textures/
assets/minecraft/font/
assets/minecraft/sounds.json
```

Server Enforcement:

- Use `PlayerResourcePackStatusEvent` to monitor pack acceptance
- Optionally kick or restrict players who decline the resource pack
- Geyser supports resource packs (critical for Bedrock compatibility)

8.3.2 Phase 2: Custom Item System (Core Abstraction)

This forms the foundation for all custom content.

Design: Create a wrapper class structure:

```
class CustomItem {
  String id;
  Material baseMaterial;
  int customModelData;
  Consumer<PlayerInteractEvent> onUse;
}
```

Implementation:

- Store ID in `PersistentDataContainer` using a **NamespacedKey** to prevent collisions with other plugins
- Validate items on interaction by checking the persistent data container
- Block vanilla behavior when necessary by canceling events

Example Implementation:

```
/**
 * Simple check for custom item identification.
 */
@EventHandler
public void onInteract(PlayerInteractEvent event) {
    ItemStack item = event.getItem();
```

```
    if (item == null || item.getType() == Material.AIR) return;

    PersistentDataContainer pdc = item.getItemMeta()
        .getPersistentDataContainer();
    NamespacedKey customItemKey = new NamespacedKey(plugin, "custom_item_id");

    if (pdc.has(customItemKey, PersistentDataType.STRING)) {
        handleCustomAction(event);
    }
}
```

Result: This system enables custom tools, fish items, furniture items, and machine items, effectively replacing "mod items" within the **plugin architecture**.

8.3.3 Phase 3: Fake Blocks and Furniture

Techniques: Choose implementation based on complexity:

- **Simple furniture**: Invisible armor stands
- **Advanced**: Display entities (Minecraft 1.19+)
- **Alternative**: Item frames (fixed + invisible)

Interaction:

- Ray trace from player position
- Match entity UUID
- Trigger custom logic based on interaction

Persistence:

- Store locations in SQLite, YAML, or JSON
- Re-spawn entities on server start

Example Use Cases: Chairs, tables, lamps, aquariums

8.3.4 Phase 4: Animals and Fish

How This Works: Use vanilla mobs with modifications:

- Custom name (hidden)
- Custom model via resource pack
- AI tweaks
- Metadata tags

Fish Implementation:

- Use vanilla fish types: Cod, Salmon, Tropical fish
- Override model through resource pack
- Implement biome-based spawning logic

Animals: Implement a variant system:

```
enum AnimalVariant {
  RED_FOX,
  WHITE_FOX,
  BLUE_FOX
}
```

Behavior Customization:

- Cancel breeding
- Custom drops
- Custom sounds

8.3.5 Phase 5: Fake Machines

This is where Paper's plugin architecture excels.
Concept: Machine = state machine: `IDLE` → `PROCESSING` → `OUTPUT`
Example Machine (Furnace-like Device):

- Input slot (GUI)
- Timer mechanism
- Output slot
- Animation via model swap or sound

Storage Requirements:

- Block location
- Machine type
- Current state
- Timer progress

Implementation:

- Inventory GUI for interaction
- Scheduled tasks for processing
- Cancel vanilla block interaction

8.3.6 Phase 6: GUI Framework

Java Edition (Paper):

- Inventory GUIs
- Click handling
- Pagination
- Animations (slot swapping)

Bedrock Edition (Geyser):

- Forms (simple, modal, custom)
- Map GUIs to forms where possible

- Fallback to inventory GUIs if needed
- **Important**: For full access to Bedrock-specific form features and player metadata, ensure **Geyser** is on the backend or use the **Geyser API** (see Section 5.2.2 for placement considerations)

Abstraction:

```
interface Menu {
  void open(Player p);
}
```

8.3.7 Phase 7: Input and Interaction Engine

Available Inputs:

- Right click (via `PlayerInteractEvent`)
- Left click (via `PlayerInteractEvent`)
- Sneak (via player metadata or movement events)
- Sprint (via player metadata or movement events)
- Swap hand (via `PlayerSwapHandItemsEvent`)
- Drop key (detected **server-side** via `PlayerDropItemEvent`, which can be canceled to trigger custom actions instead of dropping the item)

Input Combination: Example: `Sneak + RightClick + CustomItem = Action`
Vanilla Behavior Control:

- Cancel interaction events (e.g., `PlayerInteractEvent.setCancelled(true)`)
- Cancel drop events to intercept item drops (e.g., `PlayerDropItemEvent.setCancelled(true)`)
- Replace behavior fully with custom logic

8.3.8 Phase 8: Fonts and Polish

Custom Fonts:

- Unicode private range
- Icons in GUIs
- Text-based progress bars
- Decorative UI elements

Sounds:

- Contextual sounds
- Machine loops
- Fish splashes
- UI feedback

8.4 Tech Stack Recommendations

8.4.1 Libraries and APIs

- **Paper API**: Core server API (required)
- **Adventure**: Text formatting and font handling
- **ProtocolLib**: Optional but powerful for advanced packet manipulation
- **Citizens**: NPC dialogs and interactions
- **Geyser API**: Bedrock client detection and form handling

8.4.2 Storage Solutions

- **SQLite**: For machines and furniture persistence
- **YAML**: For configuration files
- **JSON**: For content definitions

8.5 Development Strategy

Start Small:

- One custom item
- One GUI
- One machine
- One fish variant

Then Generalize:

- Data-driven configurations
- Reusable systems
- Content packs without recompiling

8.6 Goals and Constraints

This approach works perfectly for the following goals:

1. Zero client setup
2. Java + Bedrock compatibility
3. Stability
4. Hosting at scale

This approach does not require:

1. Real new blocks
2. True modded gameplay
3. Client performance boosts

8.7 Conclusion

This implementation plan demonstrates that achieving mod-like experiences with **Paper and plugins** is not only feasible but represents the standard approach for large-scale **cross-play servers**. By following the content illusion system methodology and implementing the phases sequentially, you can deliver rich custom content while maintaining zero client setup requirements and full **compatibility with both Java and Bedrock editions**.